

# EasyVisa Project

## Post Graduate Program in Artificial Intelligence and Machine Learning

Prepared by Christian Caceres

February 21, 2026

# Contents / Agenda

- Executive Summary
- Business Problem Overview and Solution Approach
- EDA Observations
- Data Preprocessing
- Model Performance Strategy and Comparison
- Hyperparameter Tuning
- Final Model Selection
- Conclusions and Recommendations
- Appendix

# Executive Summary

The EasyVisa Machine Learning project was designed to predict whether a work visa application will be certified or denied prior to submission. The primary objective is to reduce approval risk, minimize unexpected denials, and provide smarter, data-driven guidance to clients.

During data analysis, several important patterns were identified. Higher education levels and prior job experience were associated with higher certification rates. Full-time roles were more likely to be approved. Prevailing wage levels influenced outcomes, and certain regions and employer characteristics played a meaningful role in certification decisions.

After evaluating multiple ensemble models, the **tuned XGBoost model** was selected due to its strong performance, achieving 94.7% recall and an 81.5% F1-score. Its high recall directly aligns with the business objective of minimizing high-risk visa approvals while maintaining balanced predictive performance.

Overall, this solution enables EasyVisa to proactively assess application risk, improve decision-making, optimize operational resources, and enhance client success rates.

# Business Problem Overview and Solution Approach

## Problem Overview:

EasyVisa provides end-to-end support to organizations and international talent navigating the U.S. work visa process. Visa outcomes are influenced by a wide range of variables, including candidate qualifications, wage compliance, employer characteristics, and regional labor market conditions, making the process both data-intensive and highly regulated. As a result, high-risk applications may be submitted without proactive risk mitigation, increasing the likelihood of unexpected denials for clients.

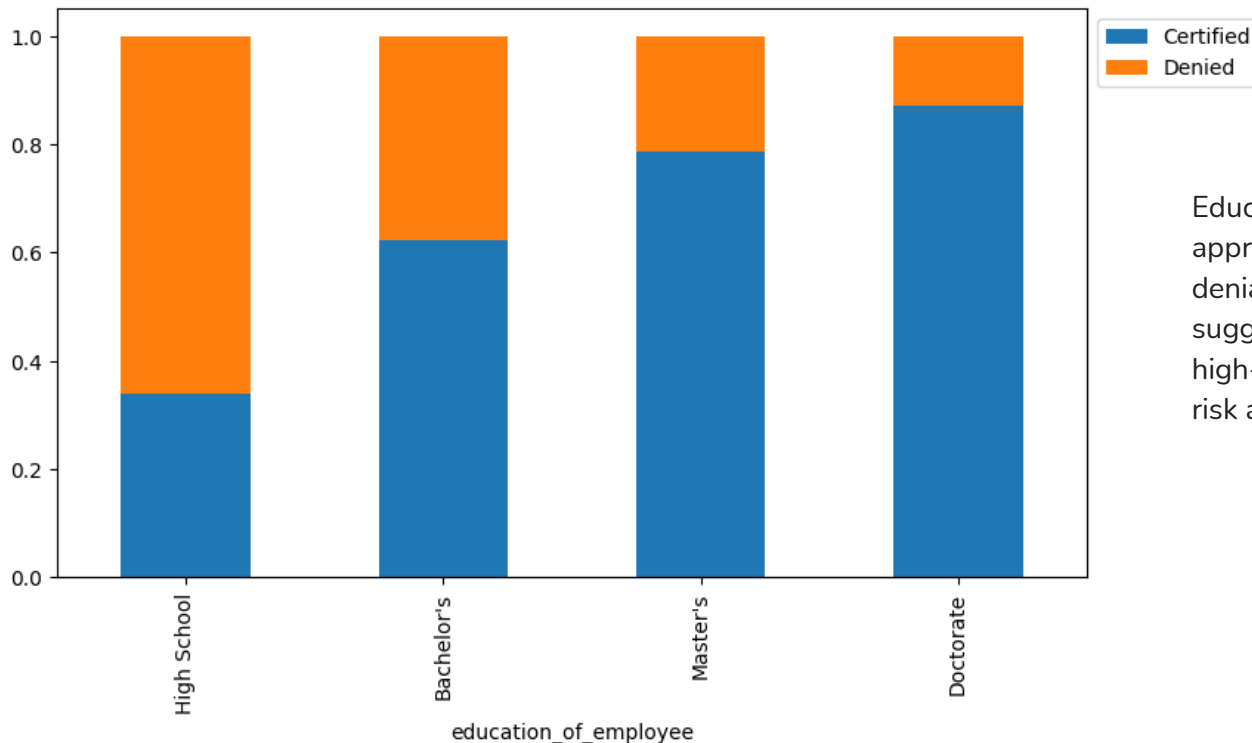
## Solution Approach:

To address the problem, EasyVisa will develop a machine learning classification model that predicts whether a visa application is likely to be approved or denied based on historical data. The approach includes analyzing and preparing the data, building and comparing multiple predictive models, and selecting the best-performing model using evaluation metrics such as recall, precision, and F1-score. The final solution will provide actionable insights into key approval drivers and enable proactive risk assessment, helping improve decision-making, increase approval rates, and optimize operational efficiency.

# EDA Observations

Does education have any impact on visa certification outcomes?

## Education Level / Case Status

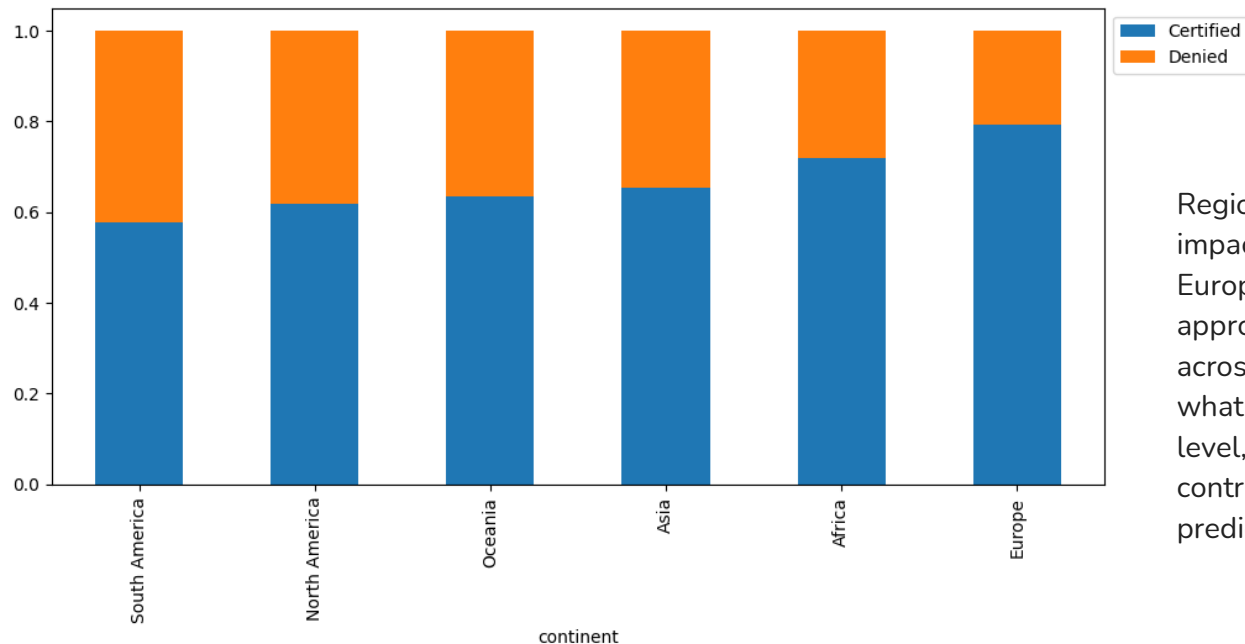


Education level is a major predictor of visa approval. As educational attainment increases, denial risk decreases significantly. This suggests education should be treated as a high-impact feature in predictive modeling and risk assessment for visa applications.

# EDA Observations

## How the visa status vary across different continents?

Region / Case Status

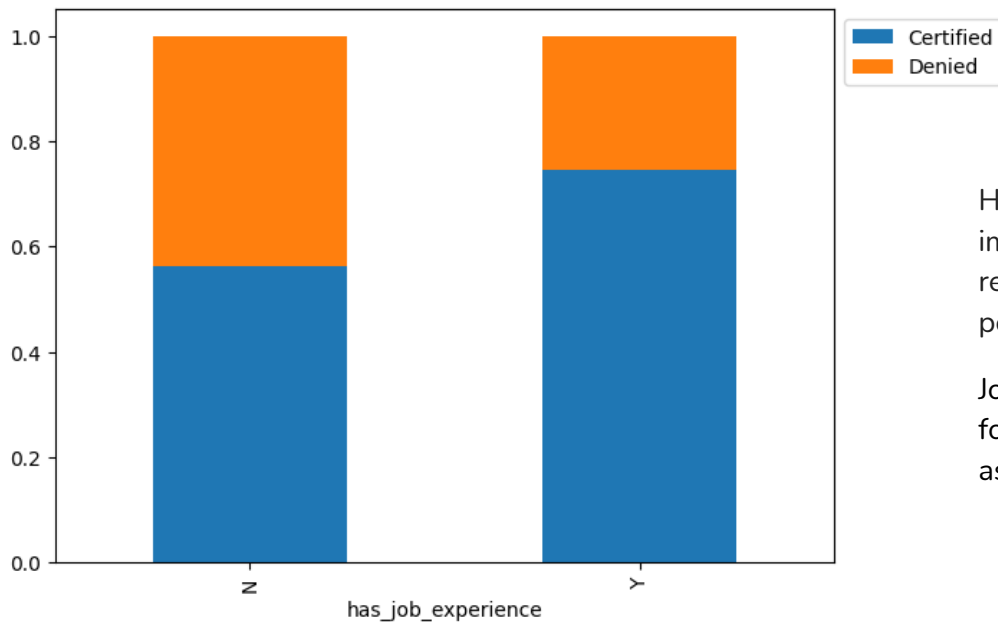


Region of origin appears to have a moderate impact on certification outcomes, with European applicants experiencing the highest approval likelihood. However, the variation across continents is less pronounced than what is typically observed with education level, indicating that continent may be a contributing factor but not the strongest predictor of approval.

# EDA Observations

Does having prior work experience influence the likelihood of visa certification?

## Job Experience / Case Status



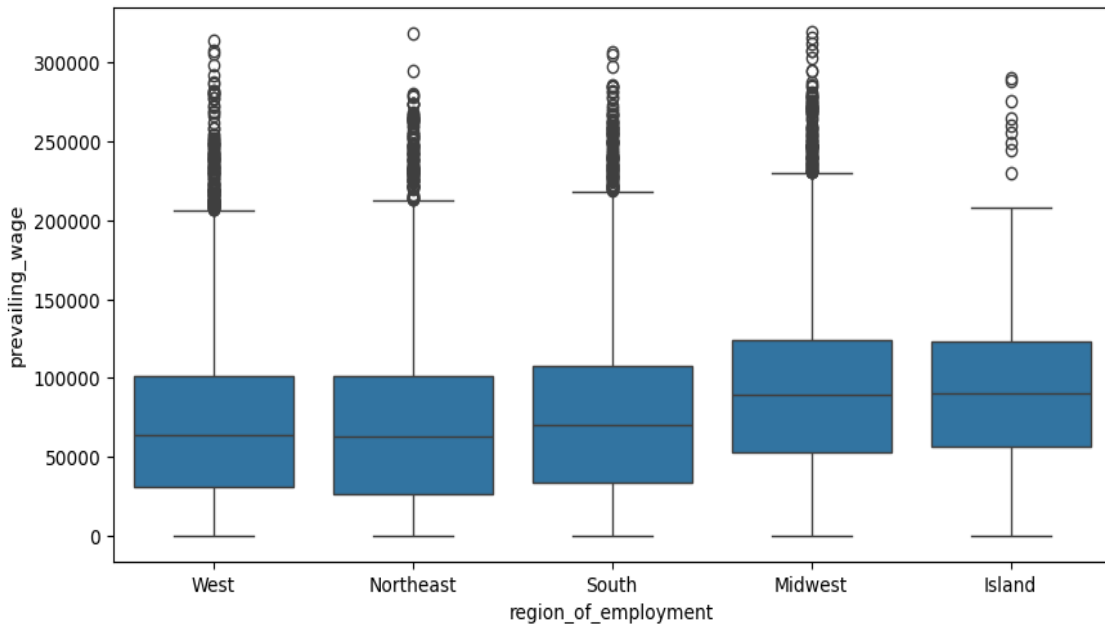
Having prior job experience materially improves the probability of certification and reduces denial risk. From a modeling perspective

Job experience is likely an important feature for predicting application outcomes and assessing approval probability

# EDA Observations

Is the prevailing wage similar across all the regions of the US?

## Region of Employment / Prevailing Wage



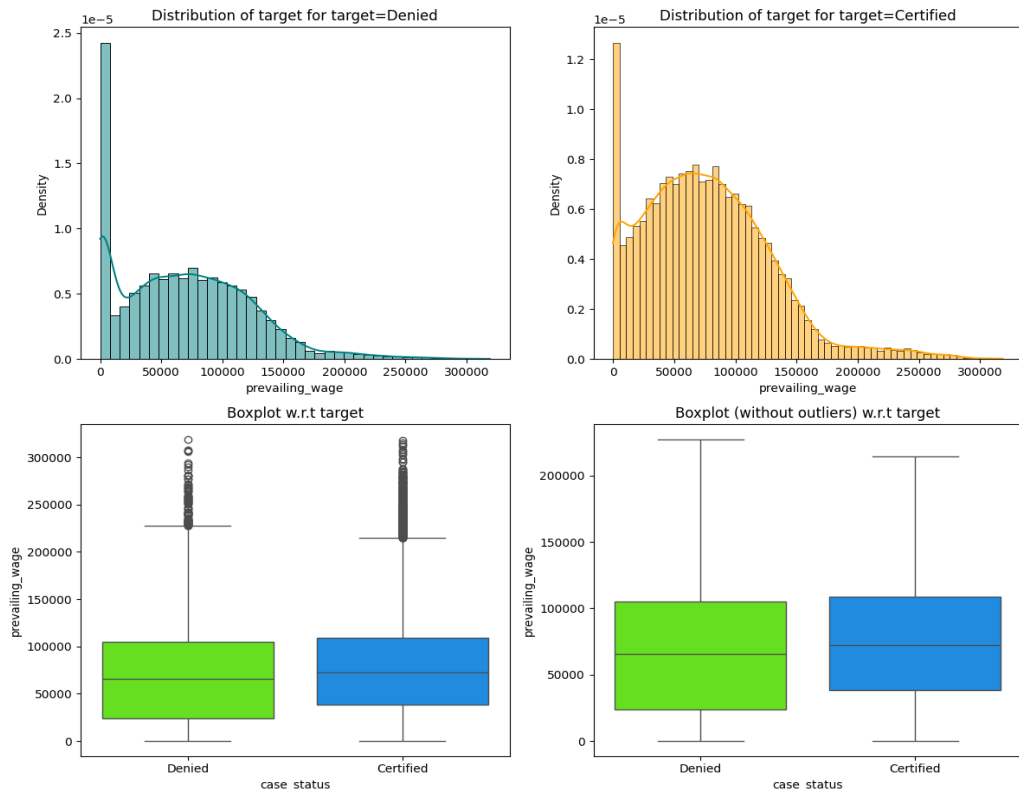
The prevailing wage varies across regions, with the **Midwest and Island** showing the highest median wages, followed by the **South**, while the **West and Northeast** have relatively lower median values. All regions display a wide spread of wages and significant high-end outliers, indicating the presence of some very high-paying roles



# EDA Observations

Does the visa status change based on the prevailing wage?

## Prevailing Wage / Case Status



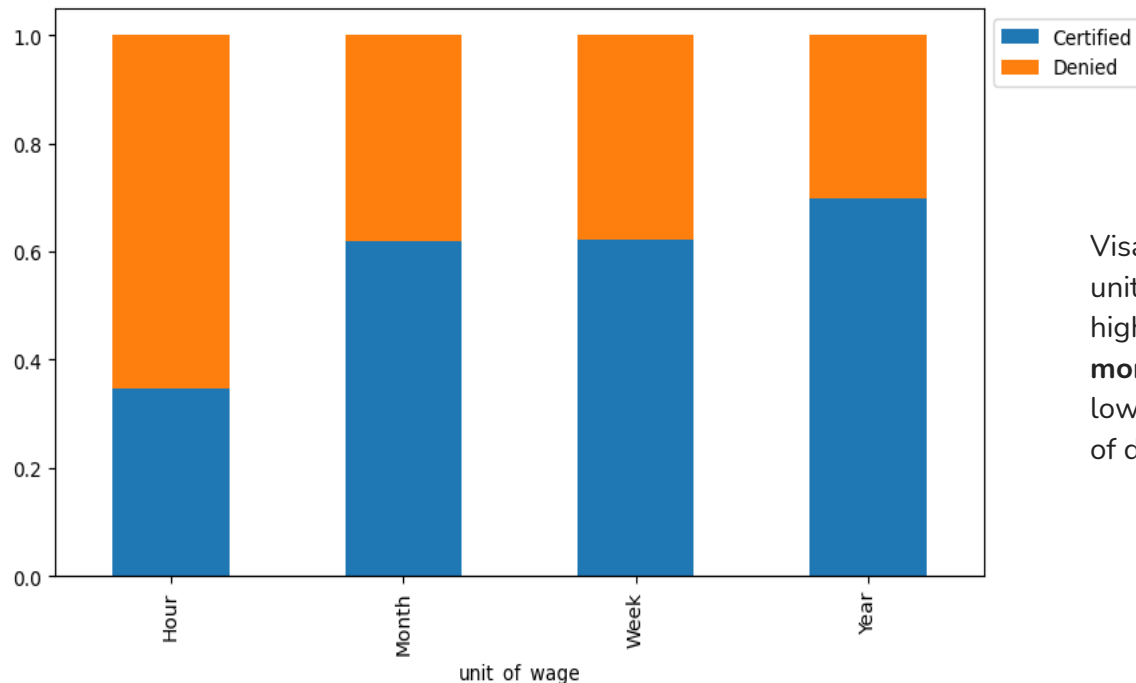
Certified visa applications tend to have higher prevailing wages compared to denied cases. The distribution for certified cases is slightly shifted toward higher salary ranges, and this difference remains even after removing outliers.

Both groups show right-skewed distributions with some very high-wage outliers, but the overall central tendency (median) is higher for certified applications.

# EDA Observations

Does unit of wage have any impact on visa applications getting certified?

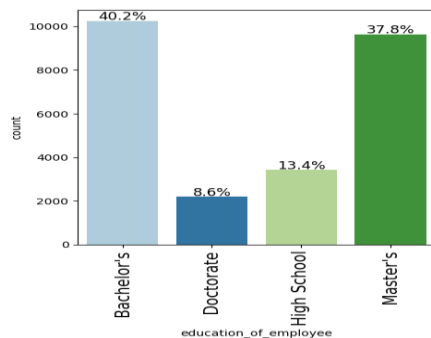
## Unit of Wage / Case Status



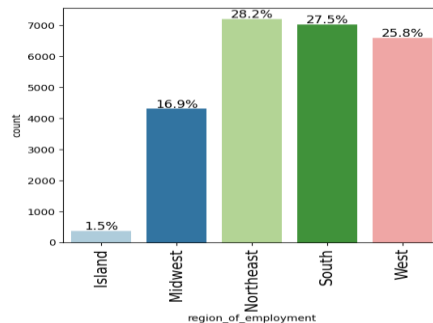
Visa certification rates vary significantly by wage unit. Applications with **yearly wages** have the highest approval rate, followed by **weekly and monthly wages**, while **hourly wages** have the lowest certification rate and the highest proportion of denials.

# EDA Observations

## Education Level



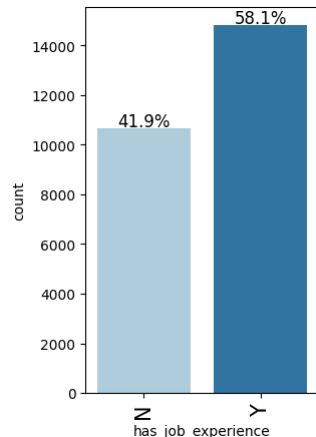
## Region of Employment



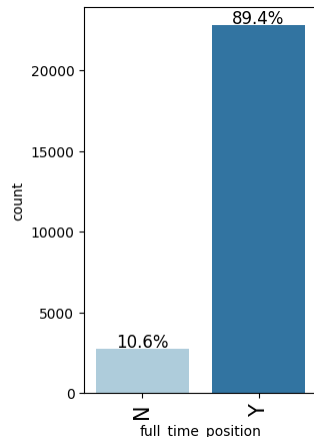
**Education:** Nearly 78% of applicants hold at least a Bachelor's degree. The dataset is highly skewed toward higher education levels.

**Employment:** Applications are fairly evenly distributed among Northeast, South, and West - Employment demand for foreign workers appears strongest in Northeast, South and West

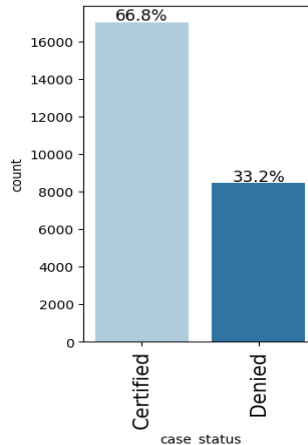
## Job Experience



## Employment



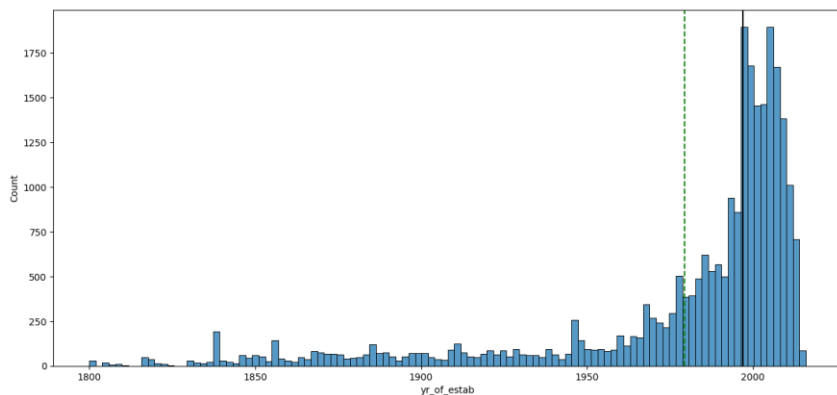
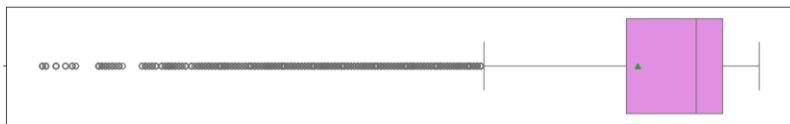
## Case Approval



Approximately **58% of applicants have prior job experience**, and nearly **90% of positions are full-time**, reflecting the structured nature of employment-based visa sponsorship. In terms of outcomes, about **67% of cases are certified**, while **33% are denied**, creating a moderate class imbalance in the target variable.

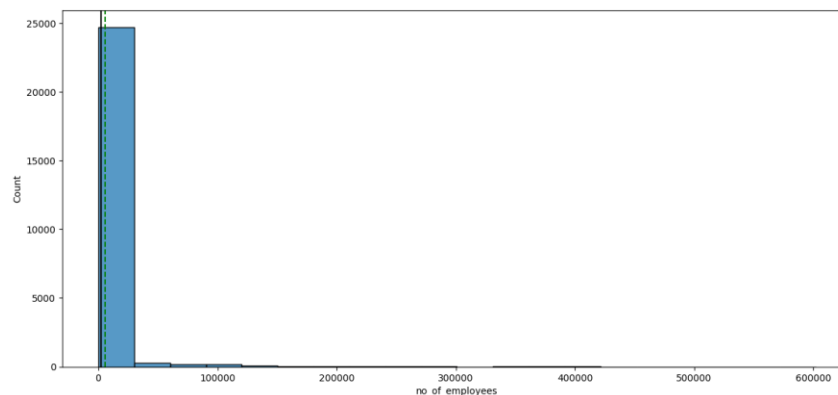
# EDA Observations

## Company Years of Establishment



- The distribution is heavily left-skewed
- Most companies were established **after 1970**, with a strong concentration between **1990–2015**.
- The median appears to be in the **late 1990s or early 2000s**, indicating most sponsoring companies are relatively modern.

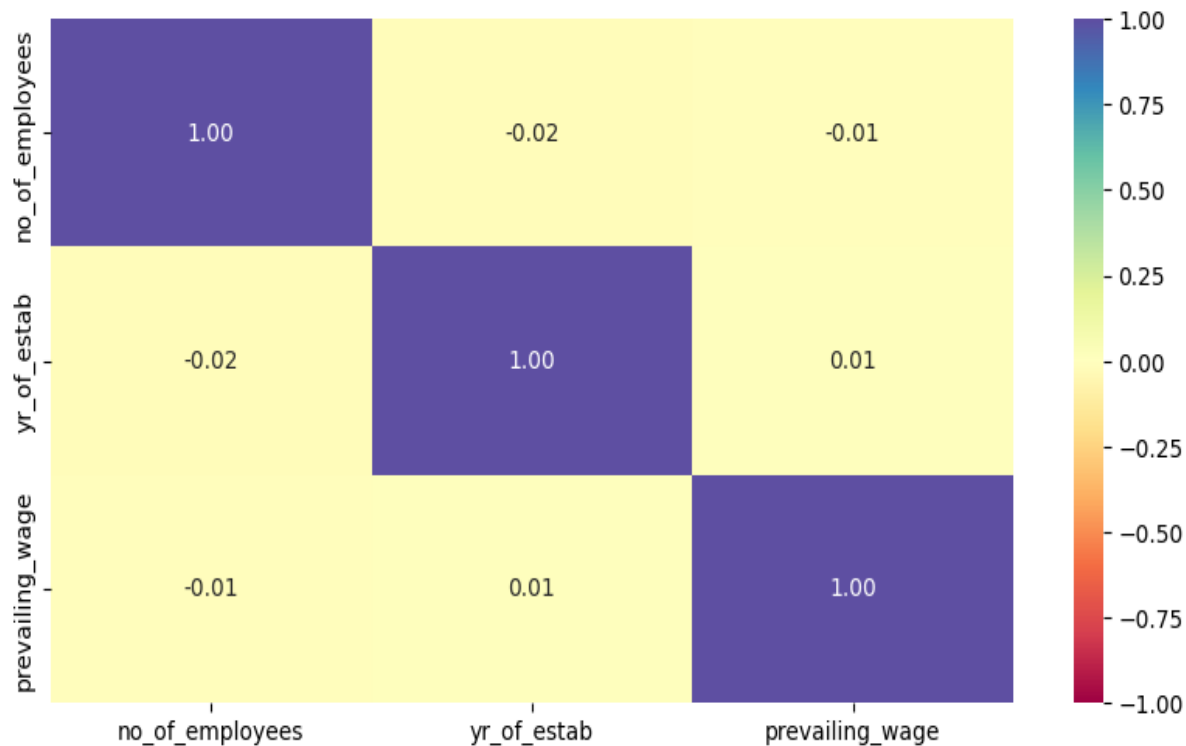
## Number of Employees



- The vast majority of companies have **relatively small employee counts**.
- A small number of companies have **very large employee sizes (100K–600K+)**, creating strong outliers.
- The median is much lower than the mean, confirming heavy skewness.

# EDA Observations

Correlation Heat Map



There is **no significant linear correlation** among the numerical features, meaning each variable likely contributes independent information to the predictive model.

# Data Preprocessing: Data Types

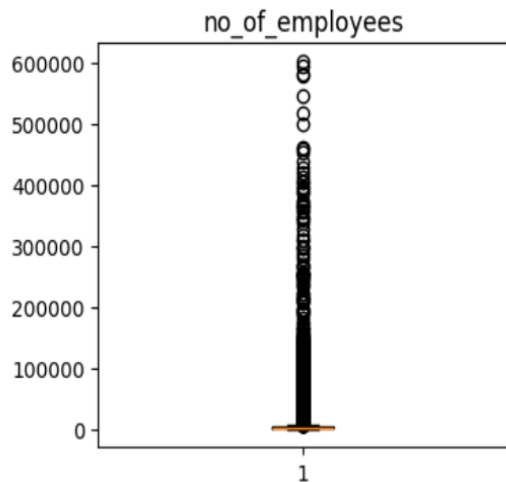
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25480 entries, 0 to 25479
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   case_id                               25480 non-null  object
1   continent                             25480 non-null  object
2   education_of_employee                 25480 non-null  object
3   has_job_experience                    25480 non-null  object
4   requires_job_training                 25480 non-null  object
5   no_of_employees                       25480 non-null  int64
6   yr_of_estab                           25480 non-null  int64
7   region_of_employment                  25480 non-null  object
8   prevailing_wage                       25480 non-null  float64
9   unit_of_wage                          25480 non-null  object
10  full_time_position                    25480 non-null  object
11  case_status                           25480 non-null  object
dtypes: float64(1), int64(2), object(9)
memory usage: 2.3+ MB
```

## Data Type

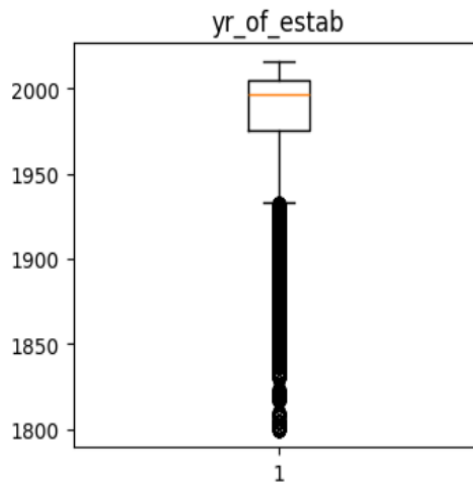
- Int 64: 2 columns (numerical)
- Float 64: 1 column (numerical)
- Object: 9 columns (numerical)
- Columns: 12
- Rows: 25480
- No missing values
- 'case\_id' should be dropped
- Target variable: 'case\_status'

# Data Preprocessing: Outlier Detection

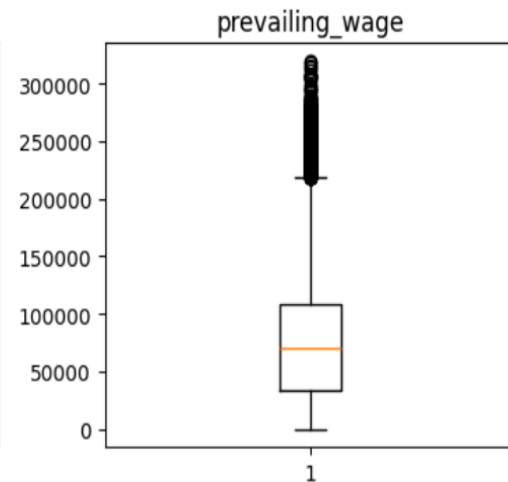
## Outlier detection using Boxplot



- Large number of outliers
- Extremely right-skewed distribution
- High Variability



- Left Skewness
- Several lower end outliers
- High Variability



- Extremely right-skewed distribution
- High Variability

# Data Preprocessing: Duplicate / Missing Values

## Duplicate Values

Checking for duplicate values

```
# checking for duplicate values
data.duplicated().sum() ## Complete the code to check for duplicate values

np.int64(0)
```



There are no duplicate values

## Missing Values

Checking for missing values

```
data.isnull().sum() ## Complete the code to check for missing values

...
0
case_id 0
continent 0
education_of_employee 0
has_job_experience 0
requires_job_training 0
no_of_employees 0
yr_of_estab 0
region_of_employment 0
prevailing_wage 0
unit_of_wage 0
full_time_position 0
case_status 0

dtype: int64
```



There are no missing values



# Data Preprocessing: Feature Engineering

## Encoding categorical features

```
data["case_status"] = data["case_status"].apply(lambda x: 1 if x == "Certified" else 0)

X = data.drop(["case_status"], axis=1)
y = data["case_status"]

X = pd.get_dummies(X, drop_first=True)
# X = X.astype(float)
```

Target variable '**case\_status**' was encoded to numerical values to allow for the ML model to process the information:

- Certified: 1
- Denied: 0

## Two Step Data Split

```
# Splitting data into training, validation and test set:
# first we split data into 2 parts, say temporary and test

X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1, stratify=y
)

# then we split the temporary set into train and validation

X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=1, stratify=y_temp
)
```

The data set was split into 20% test and 80% training

The train data set was then split into 20% validation and 80% left for training

Validation data is required to help tune the model without touching the test data thus preventing data leaks.

- ❑ Training Set: (15288, 21)
- ❑ Validation Set: (5096, 21)
- ❑ Test Set: (5096, 21)

# Model Performance Strategy and Comparison

| Data         | Training                                                                                                                                                                                                                                           | Validation                                                                                                                                                                                                                                               | Sampling Strategy                                                                | Training Sample case_status                                                               |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Original     | <ul style="list-style-type: none"> <li>Bagging: 0.989150179193873</li> <li>Random Forest: 1.0</li> <li><b>Gradient Boosting: 0.8291218182658106</b></li> <li>Adaboost: 0.8204269947530306</li> <li>XGBoost: 0.8963229453814886</li> </ul>          | <ul style="list-style-type: none"> <li>Bagging: 0.7736516357206012</li> <li>Random Forest: 0.8049978941457251</li> <li><b>Gradient Boosting: 0.826637008202419</b></li> <li>Adaboost: 0.8180081855388813</li> <li>XGBoost: 0.8079119654547987</li> </ul> | N/A                                                                              | <ul style="list-style-type: none"> <li>Certified: 10210</li> <li>Denied: 5078</li> </ul>  |
| Oversampled  | <ul style="list-style-type: none"> <li>Bagging: 0.9875473741201949</li> <li>Random Forest: 0.9999510260051913</li> <li>Gradient Boosting: 0.8072434234901815</li> <li>Adaboost: 0.8005498403689252</li> <li>XGBoost: 0.8708686342053813</li> </ul> | <ul style="list-style-type: none"> <li>Bagging: 0.7665171898355755</li> <li>Random Forest: 0.7965442764578834</li> <li>Gradient Boosting: 0.8173049645390071</li> <li>Adaboost: 0.8195334879279771</li> <li>XGBoost: 0.8129304286718201</li> </ul>       | <b>SMOTE</b><br>Artificially increases the minority class samples : 'denied'     | <ul style="list-style-type: none"> <li>Certified: 10210</li> <li>Denied: 10210</li> </ul> |
| Undersampled | <ul style="list-style-type: none"> <li>Bagging: 0.9803687095166915</li> <li>Random Forest: 1.0</li> <li>Gradient Boosting: 0.7281441717791411</li> <li>Adaboost: 0.7015343047380103</li> <li>XGBoost: 0.8720351390922401</li> </ul>                | <ul style="list-style-type: none"> <li>Bagging: 0.7057046979865772</li> <li>Random Forest: 0.7417218543046358</li> <li>Gradient Boosting: 0.776595744680851</li> <li>Adaboost: 0.765990884802766</li> <li>XGBoost: 0.7459304181295883</li> </ul>         | <b>Random Under Sample</b><br>Decreases the majority class samples : 'certified' | <ul style="list-style-type: none"> <li>Certified: 5078</li> <li>Denied: 5078</li> </ul>   |

This comparison evaluates five ensemble models (Bagging, Random Forest, Gradient Boosting, AdaBoost, XGBoost) across three data strategies: **Original, Oversampled, and Undersampled**, focusing on training vs validation performance.

**Gradient Boosting on Original Data** appears to be the strongest overall choice.

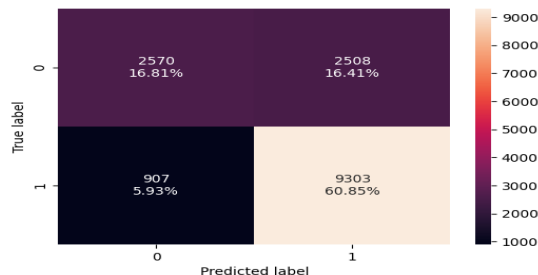
# Hyperparameter Tuning: Random Forest (rf\_tuned)

Training

## Tuning

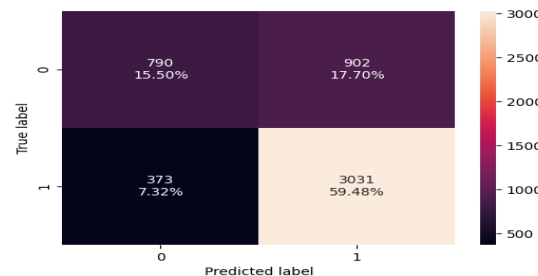
```
parameters = {
    "max_depth": list(np.arange(5, 15, 5)),
    "max_features": ["sqrt", "log2"],
    "min_samples_split": [3, 5, 7],
    "n_estimators": np.arange(10, 40, 10),
}
```

## Confusion Matrix



Validation

```
parameters = {
    "max_depth": list(np.arange(5, 15, 5)),
    "max_features": ["sqrt", "log2"],
    "min_samples_split": [3, 5, 7],
    "n_estimators": np.arange(10, 40, 10),
}
```



## Performance Metrics

- Accuracy: 0.77
- Recall: 0.91
- Precision: 0.78
- F1: 0.84

- Accuracy: 0.74
- Recall: 0.89
- Precision: 0.77
- F1: 0.82

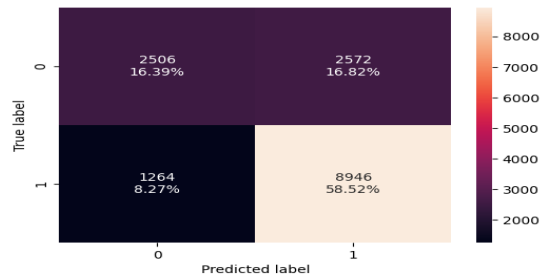
The tuned Random Forest model demonstrates strong validation performance, achieving high recall (89%) and a balanced F1-score (82%). This indicates effective detection of the target class while maintaining reasonable precision, suggesting good generalization.

# Hyperparameter Tuning: AdaBoost (abc\_tuned)

## Tuning

```
parameters = {
    # Let's try different max_depth for base_estimator
    "estimator": [
        DecisionTreeClassifier(max_depth=2, random_state=1),
        DecisionTreeClassifier(max_depth=3, random_state=1),
    ],
    "n_estimators": np.arange(50,110,25),
    "learning_rate": np.arange(0.01,0.1,0.05),
}
```

## Confusion Matrix

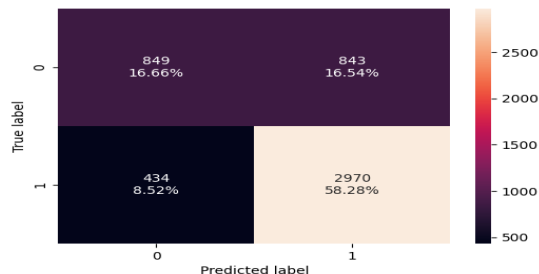


## Performance Metrics

- Accuracy: 0.74
- Recall: 0.87
- Precision: 0.77
- F1: 0.82

Training

```
parameters = {
    # Let's try different max_depth for base_estimator
    "estimator": [
        DecisionTreeClassifier(max_depth=2, random_state=1),
        DecisionTreeClassifier(max_depth=3, random_state=1),
    ],
    "n_estimators": np.arange(50,110,25),
    "learning_rate": np.arange(0.01,0.1,0.05),
}
```



- Accuracy: 0.74
- Recall: 0.87
- Precision: 0.77
- F1: 0.82

Validation

The tuned AdaBoost model shows solid validation results with strong recall (87%) and balanced performance (82%) making it a viable model for final testing and comparison.

# Hyperparameter Tuning: Gradient Boosting (gbc\_tuned)

## Tuning

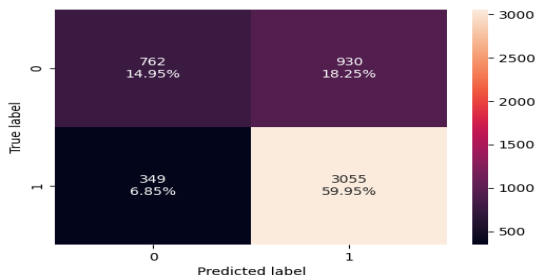
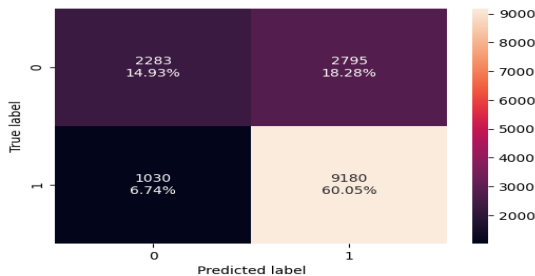
Training

```
parameters = {
    "n_estimators": np.arange(50,110,25),
    "subsample": [0.7,0.9],
    "max_features": [0.7, 0.8, 0.9, 1],
    "learning_rate": [0.01,0.1,0.05],
```

Validation

```
parameters = {
    "n_estimators": np.arange(50,110,25),
    "subsample": [0.7,0.9],
    "max_features": [0.7, 0.8, 0.9, 1],
    "learning_rate": [0.01,0.1,0.05],
```

## Confusion Matrix



## Performance Metrics

- Accuracy: 0.74
- Recall: 0.89
- Precision: 0.76
- F1: 0.82

- Accuracy: 0.74
- Recall: 0.89
- Precision: 0.76
- F1: 0.82

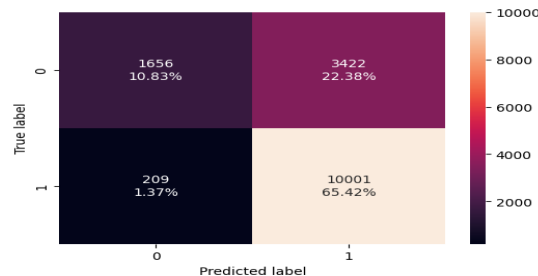
The tuned Gradient Boosting model demonstrates the strongest validation performance, achieving the high recall (89%) and F1-score (82%) suggesting good generalization.

# Hyperparameter Tuning: XGBoost Classifier (xgb\_tuned)

## Tuning

```
parameters = {
    "n_estimators": np.arange(50,110,25),
    "scale_pos_weight": [1,2,5],
    "subsample": [0.9, 1],
    "learning_rate": [0.01,0.1,0.05],
    "gamma": [1,3]
```

## Confusion Matrix

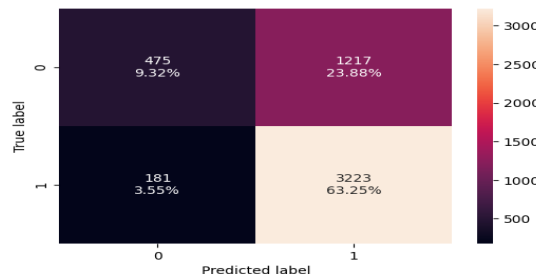


## Performance Metrics

- Accuracy: 0.76
- Recall: 0.97
- Precision: 0.74
- F1: 0.84

Training

```
parameters = {
    "n_estimators": np.arange(50,110,25),
    "scale_pos_weight": [1,2,5],
    "subsample": [0.9, 1],
    "learning_rate": [0.01,0.1,0.05],
    "gamma": [1,3]
```



- Accuracy: 0.72
- Recall: 0.94
- Precision: 0.72
- F1: 0.82

Validation

The tuned XGBoost model achieves the highest recall (94%), making it highly effective at identifying positive cases. This aligns directly with the project objective to minimize false negatives.

# Final Model Selection

Training performance comparison:

|           | Tuned Random Forest | Tuned Adaboost Classifier | Tuned Gradient Boost Classifier | Best Performing Model<br>XGBoost Classifier Tuned |
|-----------|---------------------|---------------------------|---------------------------------|---------------------------------------------------|
| Accuracy  | 0.776622            | 0.749084                  | 0.749804                        | 0.762493                                          |
| Recall    | 0.911166            | 0.876200                  | 0.899119                        | 0.979530                                          |
| Precision | 0.787656            | 0.776697                  | 0.766597                        | 0.745064                                          |
| F1        | 0.844921            | 0.823454                  | 0.827586                        | 0.846359                                          |

Validation performance comparison:

|           | Tuned Random Forest | Tuned Adaboost Classifier | Tuned Gradient Boost Classifier | Best Performing Model<br>XGBoost Classifier Tuned |
|-----------|---------------------|---------------------------|---------------------------------|---------------------------------------------------|
| Accuracy  | 0.749804            | 0.749411                  | 0.749019                        | 0.725667                                          |
| Recall    | 0.890423            | 0.872503                  | 0.897474                        | 0.946827                                          |
| Precision | 0.770659            | 0.778914                  | 0.766625                        | 0.725901                                          |
| F1        | 0.826223            | 0.823057                  | 0.826905                        | 0.821775                                          |

## Model Objective

High Recall = Fewer False Negatives

Business Objective is to minimize unexpected denials

High Recall = Fewer False Negatives

Business Objective is to minimize unexpected denials

**The tuned XGBoost is the best model** because it achieves the highest Recall (94%) , aligning directly with the business objective of minimizing high-risk visa approvals while maintaining a strong F1-score balance.

# Best Model: XGBoost Classifier (xgb\_tuned)

## Training Data Results

|   | Accuracy | Recall   | Precision | F1       |
|---|----------|----------|-----------|----------|
| 0 | 0.713108 | 0.947121 | 0.71549   | 0.815171 |

The tuned **XGBoost** model was selected due to its strong performance, achieving 94.7% recall and an F1 score of 81.5%.

It is optimized to prioritize identifying high-risk visa applications, minimizing missed denials.

Most relevant features include:

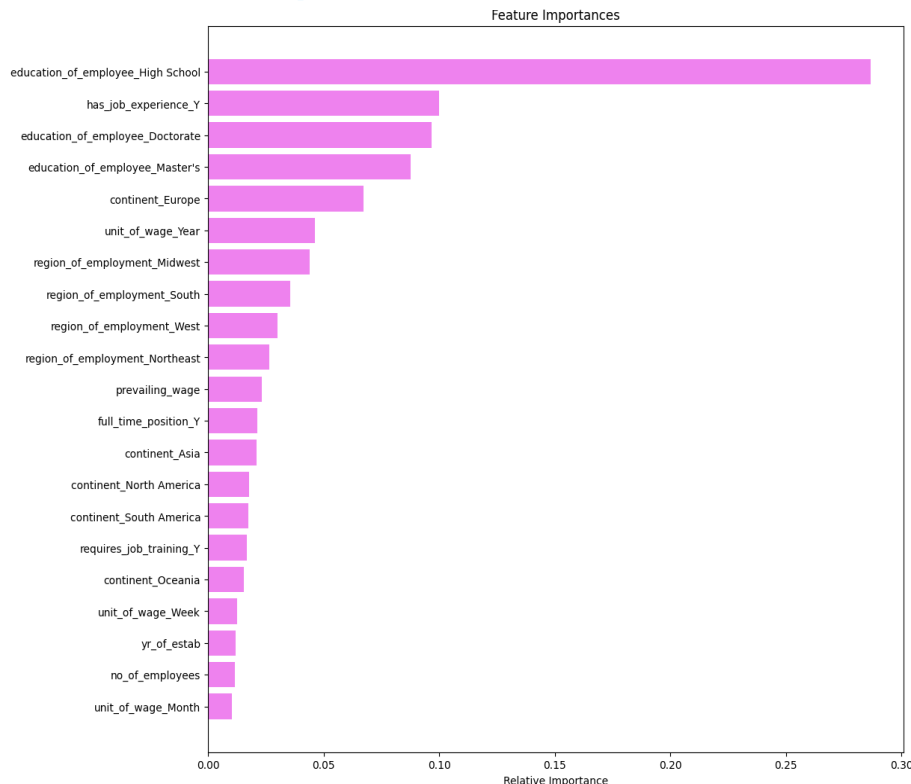
- Education
- Job Experience
- Wage Level
- Geographic Region

## Tuned Parameters

Optimal Parameters Include:

- ✓ n\_estimator = 100
- ✓ learning\_rate = 0.1
- ✓ gamma = 1
- ✓ subsample = 1 <default>

## Feature Importance





# Conclusions & Recommendations

The EasyVisa predictive model successfully addresses the core challenge of identifying high-risk visa applications before submission. Key factors such as education, job experience, employment type, wage level, region, and employer characteristics significantly influence certification outcomes. Among the models tested, the tuned XGBoost model delivered the strongest performance, achieving 94.7% recall and an 81.5% F1-score, effectively minimizing high-risk approvals while maintaining balanced performance.

## Recommendations:

- **Model Deployment:** Integrate the tuned XGBoost model into the pre-submission workflow.
- **Business Integration:** Focus on wage compliance, job structuring, and qualifying positioning.
- **Performance Monitoring:** Continuously track key performance metrics—including recall, precision, and overall model effectiveness—and incorporate periodic retraining into the model lifecycle to maintain accuracy, manage data drift and ensure sustained performance.
- **Data Enhancement:** Incorporate relevant external data and refine the features to improve predictive accuracy.
- **Approval process:** Implement structure to risk categories (High, Medium, Low) to prioritize the approval process.

# APPENDIX

# Data Background and Contents

## Statistical Summary

|                 | count   | mean         | std          | min       | 25%      | 50%      | 75%         | max       |
|-----------------|---------|--------------|--------------|-----------|----------|----------|-------------|-----------|
| no_of_employees | 25480.0 | 5667.043210  | 22877.928848 | -26.0000  | 1022.00  | 2109.00  | 3504.0000   | 602069.00 |
| yr_of_estab     | 25480.0 | 1979.409929  | 42.366929    | 1800.0000 | 1976.00  | 1997.00  | 2005.0000   | 2016.00   |
| prevailing_wage | 25480.0 | 74455.814592 | 52815.942327 | 2.1367    | 34015.48 | 70308.21 | 107735.5125 | 319210.27 |

The dataset contains 25,480 observations with key numerical variables including number of employees, year of establishment, and prevailing wage.

- The **number of employees** variable shows strong right skewness with extreme outliers and at least one invalid negative value.
- The **year of establishment** appears reasonable with most companies founded after the 1970s.
- The **prevailing wage** variable shows moderate variability with a wide range and a few unusually low values.

Overall, the data is usable but requires minor cleaning and outlier handling before model development.



**Happy Learning !**

